

Linked List — Complete DSA Revision Notes

1. What is a Linked List?

A linked list is a linear data structure where elements ("nodes") are stored non-contiguously in memory. Each node holds data plus a pointer/reference to the next node (and previous, in doubly linked lists).

Why use it over an array?

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Access by index	$O(1)$	$O(n)$
Insert/Delete at front	$O(n)$	$O(1)$
Insert/Delete at known position	$O(n)$ shifting	$O(1)$ if node ref known
Extra memory	None	Pointer(s) per node
Cache locality	Good	Poor

Use linked lists when you need frequent insertions/deletions and don't need random access.

2. Types of Linked Lists

1. **Singly Linked List (SLL)** - each node points to the next only.
 2. **Doubly Linked List (DLL)** - each node points to next and previous.
 3. **Circular Linked List (CLL)** - last node points back to the head (singly or doubly).
-

3. Node Structure

```
python
```

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None      # for SLL
        # self.prev = None    # add for DLL

class LinkedList:
    def __init__(self):
        self.head = None
```

4. Core Operations (Singly Linked List)

4.1 Traversal — $O(n)$

python

```
def traverse(head):
    curr = head
    while curr:
        print(curr.val, end=" -> ")
        curr = curr.next
    print("None")
```

4.2 Insertion

At head — $O(1)$

python

```
def insert_at_head(head, val):
    new_node = Node(val)
    new_node.next = head
    return new_node    # new head
```

At tail — $O(n)$ ($O(1)$ if tail pointer maintained)

python

```
def insert_at_tail(head, val):
    new_node = Node(val)
    if not head:
        return new_node
    curr = head
    while curr.next:
        curr = curr.next
    curr.next = new_node
    return head
```

At position k (0-indexed) — $O(n)$

python

```
def insert_at_pos(head, val, k):
    if k == 0:
        return insert_at_head(head, val)
    curr = head
    for _ in range(k - 1):
        if not curr: return head # invalid position
        curr = curr.next
    new_node = Node(val)
    new_node.next = curr.next
    curr.next = new_node
    return head
```

4.3 Deletion

Delete head — $O(1)$

python

```
def delete_head(head):
    if not head:
        return None
    return head.next
```

Delete by value — $O(n)$

python

```
def delete_value(head, val):
    if not head:
        return None
    if head.val == val:
        return head.next
    curr = head
    while curr.next and curr.next.val != val:
        curr = curr.next
    if curr.next:
        curr.next = curr.next.next
    return head
```

Delete node given only that node (no head access) — O(1) trick

python

```
def delete_given_node(node):
    # Copy next node's data into current, then bypass next.
    # Fails only if node is the last node.
    node.val = node.next.val
    node.next = node.next.next
```

4.4 Search — O(n)

python

```
def search(head, target):
    curr = head
    idx = 0
    while curr:
        if curr.val == target:
            return idx
        curr = curr.next
        idx += 1
    return -1
```

4.5 Get Length — O(n)

python

```
def length(head):
    count = 0
    curr = head
    while curr:
        count += 1
        curr = curr.next
    return count
```

5. Key Patterns Used Across Linked List Problems

Pattern A — Two Pointer (Slow & Fast / Runner Technique)

Used for: finding middle, detecting cycles, finding nth node from end.

```
python

slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
# slow is now at the middle
```

Pattern B — Dummy Node

Simplifies edge cases (deleting head, merging lists) by avoiding null-head special-casing.

```
python

dummy = Node(-1)
dummy.next = head
# operate using dummy.next
return dummy.next
```

Pattern C — Iterative Reversal (three pointers: prev, curr, next)

```
python
```

```
def reverse(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev    # new head
```

Pattern D — Recursive Reversal

python

```
def reverse_recursive(head):
    if not head or not head.next:
        return head
    new_head = reverse_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head
```

Pattern E — Merge Two Sorted Lists

python

```
def merge(l1, l2):
    dummy = Node(-1)
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

6. Doubly Linked List (DLL)

python

```
class DNode:
    def __init__(self, val):
        self.val = val
        self.next = None
        self.prev = None
```

Insert at head — O(1)

python

```
def insert_head(head, val):
    new_node = DNode(val)
    new_node.next = head
    if head:
        head.prev = new_node
    return new_node
```

Delete a node (reference known) — O(1)

python

```
def delete_node(head, node):
    if node.prev:
        node.prev.next = node.next
    else:
        head = node.next  # deleting head
    if node.next:
        node.next.prev = node.prev
    return head
```

Reverse a DLL — O(n)

python

```
def reverse_dll(head):
    curr = head
    new_head = head
    while curr:
        curr.prev, curr.next = curr.next, curr.prev
        new_head = curr
        curr = curr.prev  # was curr.next before swap
    return new_head
```

DLL advantages: $O(1)$ deletion given a node reference, backward traversal. DLL cost: extra memory per node for `prev` pointer.

7. Circular Linked List (CLL)

Last node's `next` points to `head` instead of `None`.

Traversal

```
python
def traverse_circular(head):
    if not head:
        return
    curr = head
    while True:
        print(curr.val)
        curr = curr.next
        if curr == head:
            break
```

Insert at end — $O(n)$ ($O(1)$ if a tail pointer is kept)

```
python
def insert_end_circular(head, val):
    new_node = Node(val)
    if not head:
        new_node.next = new_node
        return new_node
    curr = head
    while curr.next != head:
        curr = curr.next
    curr.next = new_node
    new_node.next = head
    return head
```

Used in: round-robin scheduling, Josephus problem, circular buffers.

8. Cycle Detection — Floyd's Algorithm ("Tortoise and Hare")

```
python
```



```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Finding the cycle's starting node:

```
python

def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            # phase 2: reset one pointer to head
            ptr = head
            while ptr != slow:
                ptr = ptr.next
                slow = slow.next
            return ptr
    return None
```

Why it works: if the cycle starts at distance a from head and slow travels $a + b$ before meeting fast inside a cycle of length c , mathematically resetting one pointer to head and moving both at the same speed makes them meet exactly at the cycle's start.

9. Complexity Cheat Sheet

Operation	SLL	DLL	Array
Access by index	$O(n)$	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$
Insert at head	$O(1)$	$O(1)$	$O(n)$
Insert at tail	$O(n)/O(1)^*$	$O(1)^*$	$O(1)$ amortized

Operation	SLL	DLL	Array
Delete at head	O(1)	O(1)	O(n)
Delete (node known)	O(n)**	O(1)	O(n)

* O(1) if a tail pointer is maintained. ** O(n) because SLL needs the previous node; DLL has `prev` built in.

10. Practice Questions with Solutions

Q1. Reverse a Linked List

Problem: Given the head of a singly linked list, reverse it and return the new head.

Approach: Three-pointer iterative reversal (Pattern C). O(n) time, O(1) space.

python

```
def reverseList(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Q2. Detect Cycle in a Linked List

Problem: Return True if the list has a cycle. **Approach:** Floyd's cycle detection (Section 8).

O(n) time, O(1) space.

python

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    if slow == fast:
        return True
    return False
```

Q3. Find the Middle of a Linked List

Problem: Return the middle node (second middle if even length). **Approach:** Slow/fast pointers — when fast reaches the end, slow is at the middle.

python

```
def middleNode(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Q4. Merge Two Sorted Linked Lists

Problem: Merge two sorted lists into one sorted list. **Approach:** Dummy node + two pointers (Pattern E). $O(n + m)$ time.

python

```
def mergeTwoLists(l1, l2):
    dummy = tail = Node(-1)
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

Q5. Remove N-th Node From End of List

Problem: Remove the n-th node from the end in one pass. **Approach:** Two pointers with a gap of n , plus a dummy node to handle head removal.

python

```
def removeNthFromEnd(head, n):
    dummy = Node(-1)
    dummy.next = head
    fast = slow = dummy
    for _ in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Q6. Detect and Remove Cycle (find & break it)

Problem: If a cycle exists, remove it (make the list linear again). **Approach:** Find cycle start (Section 8) using Floyd's algorithm, then walk to the node just before the start and set its `next` to `None`.

```
python

def removeCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow == fast:
            break
    else:
        return head # no cycle

    ptr1 = head
    ptr2 = slow
    if ptr1 == ptr2: # cycle starts at head
        while ptr2.next != ptr1:
            ptr2 = ptr2.next
    else:
        while ptr1.next != ptr2.next:
            ptr1 = ptr1.next
            ptr2 = ptr2.next
    ptr2.next = None
    return head
```

Q7. Check if a Linked List is a Palindrome

Problem: Return True if the list reads the same forwards and backwards. **Approach:** Find middle → reverse second half → compare both halves. $O(n)$ time, $O(1)$ space.

python

```
def isPalindrome(head):
    if not head or not head.next:
        return True

    # find middle
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next

    # reverse second half
    prev = None
    curr = slow
    while curr:
        curr.next, prev, curr = prev, curr, curr.next

    # compare
    left, right = head, prev
    while right:
        # right half is shorter or equal
        if left.val != right.val:
            return False
        left, right = left.next, right.next
    return True
```

Q8. Intersection Point of Two Linked Lists

Problem: Two lists may merge at some node. Find that node (or None). **Approach:** Two pointers; when one reaches the end, redirect it to the other list's head. They meet at the intersection after at most $(lenA + lenB)$ steps (or both hit None simultaneously if no intersection).

python

```
def getIntersectionNode(headA, headB):
    a, b = headA, headB
    while a != b:
        a = a.next if a else headB
        b = b.next if b else headA
    return a    # intersection node or None
```

Q9. Add Two Numbers Represented as Linked Lists

Problem: Digits stored in reverse order, one digit per node. Return the sum as a linked list.

Approach: Simulate elementary addition with carry, using a dummy node.

python

```
def addTwoNumbers(l1, l2):
    dummy = tail = Node(-1)
    carry = 0
    while l1 or l2 or carry:
        x = l1.val if l1 else 0
        y = l2.val if l2 else 0
        total = x + y + carry
        carry = total // 10
        tail.next = Node(total % 10)
        tail = tail.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next
```

Q10. Reverse Nodes in k-Group

Problem: Reverse every k consecutive nodes. If remaining nodes < k, leave them as is.

Approach: Reverse k nodes at a time recursively/iteratively, reconnect groups.

python

```
def reverseKGroup(head, k):
    # check there are at least k nodes left
    node = head
    for _ in range(k):
        if not node:
            return head
        node = node.next

    prev, curr = None, head
    for _ in range(k):
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt

    head.next = reverseKGroup(curr, k)  # curr is start of next group
    return prev                        # prev is new head of this group
```

Q11. Flatten a Multilevel Doubly Linked List

Problem: Each node may have a `child` pointer to a separate DLL. Flatten into a single-level DLL. **Approach:** DFS — when a `child` is found, recursively flatten it, splice it in between current node and its original next, and reconnect the tail.

python

```
def flatten(head):
    curr = head
    while curr:
        if curr.child:
            nxt = curr.next
            child_head = flatten(curr.child)
            curr.next = child_head
            child_head.prev = curr
            curr.child = None
            tail = child_head
            while tail.next:
                tail = tail.next
            tail.next = nxt
            if nxt:
                nxt.prev = tail
        curr = curr.next
    return head
```

Q12. Rotate a Linked List by k Places

Problem: Rotate the list to the right by k places. **Approach:** Make it circular, find the new tail (length - k % length - 1 steps from head), break the circle there.

python


```
def rotateRight(head, k):
    if not head or not head.next:
        return head

    # get length and connect tail to head (make circular)
    length = 1
    tail = head
    while tail.next:
        tail = tail.next
        length += 1
    tail.next = head

    k = k % length
    steps_to_new_tail = length - k
    new_tail = head
    for _ in range(steps_to_new_tail - 1):
        new_tail = new_tail.next

    new_head = new_tail.next
    new_tail.next = None
    return new_head
```

Q13. Copy a Linked List with Random Pointer

Problem: Each node has `next` and `random` (points to any node or None). Deep-copy the list. **Approach:** O(n) time, O(1) extra space using interleaving technique.

python

```

def copyRandomList(head):
    if not head:
        return None

    # Step 1: interleave cloned nodes: A -> A' -> B -> B' -> ...
    curr = head
    while curr:
        clone = Node(curr.val)
        clone.next = curr.next
        curr.next = clone
        curr = clone.next

    # Step 2: assign random pointers for clones
    curr = head
    while curr:
        if curr.random:
            curr.next.random = curr.random.next
        curr = curr.next.next

    # Step 3: separate the two lists
    curr = head
    dummy = tail = Node(-1)
    while curr:
        tail.next = curr.next
        curr.next = curr.next.next
        curr = curr.next
        tail = tail.next
    return dummy.next

```

Q14. Sort a Linked List (Merge Sort)

Problem: Sort a linked list in $O(n \log n)$ time, $O(\log n)$ space (recursion stack). **Approach:** Merge sort — split at middle, recursively sort, merge (Q4's merge function).

python

```
def sortList(head):
    if not head or not head.next:
        return head

    # split into halves
    slow, fast = head, head.next
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    mid = slow.next
    slow.next = None

    left = sortList(head)
    right = sortList(mid)
    return mergeTwoLists(left, right)    # reuse Q4
```

Q15. Remove Duplicates from a Sorted Linked List

Problem: Delete duplicate values so each element appears once. **Approach:** Single pass; compare current node with next.

python

```
def deleteDuplicates(head):
    curr = head
    while curr and curr.next:
        if curr.val == curr.next.val:
            curr.next = curr.next.next
        else:
            curr = curr.next
    return head
```

Q16. Remove Duplicates from an Unsorted Linked List

Problem: Same as above but list isn't sorted. **Approach:** Hash set to track seen values. $O(n)$ time, $O(n)$ space.

python

```
def deleteDuplicatesUnsorted(head):
    seen = set()
    dummy = Node(-1)
    dummy.next = head
    prev, curr = dummy, head
    while curr:
        if curr.val in seen:
            prev.next = curr.next
        else:
            seen.add(curr.val)
            prev = curr
        curr = curr.next
    return dummy.next
```

Q17. Segregate Even and Odd Nodes (by value)

Problem: Rearrange so all even-valued nodes come before odd-valued nodes, preserving relative order. **Approach:** Build two separate lists (evens, odds), then join them.

python

```
def segregateEvenOdd(head):
    even_dummy, odd_dummy = Node(-1), Node(-1)
    even_tail, odd_tail = even_dummy, odd_dummy
    curr = head
    while curr:
        if curr.val % 2 == 0:
            even_tail.next = curr
            even_tail = curr
        else:
            odd_tail.next = curr
            odd_tail = curr
        curr = curr.next
    odd_tail.next = None
    even_tail.next = odd_dummy.next
    return even_dummy.next
```

Q18. Josephus Problem using Circular Linked List

Problem: n people in a circle, eliminate every k-th person until one remains. Find the

survivor. **Approach:** Build a circular linked list, repeatedly skip (k-1) nodes and delete the k-th.

python

```
def josephus(n, k):
    head = Node(1)
    curr = head
    for i in range(2, n + 1):
        curr.next = Node(i)
        curr = curr.next
    curr.next = head    # make circular

    curr = head
    while curr.next != curr:
        for _ in range(k - 1):
            curr = curr.next
        curr.next = curr.next.next    # eliminate next node
    return curr.val
```

Q19. Check if Two Linked Lists are Identical

Problem: Return True if both lists have the same values in the same order.

python

```
def areIdentical(headA, headB):
    while headA and headB:
        if headA.val != headB.val:
            return False
        headA, headB = headA.next, headB.next
    return headA is None and headB is None
```

Q20. Swap Nodes in Pairs

Problem: Swap every two adjacent nodes (values or nodes — do it by relinking nodes).

Approach: Dummy node + iterative pairwise pointer swap.

python

```
def swapPairs(head):
    dummy = Node(-1)
    dummy.next = head
    prev = dummy
    while prev.next and prev.next.next:
        first = prev.next
        second = first.next
        first.next = second.next
        second.next = first
        prev.next = second
        prev = first
    return dummy.next
```

11. Common Mistakes to Avoid

- Forgetting to update `head` when deleting/inserting at the front.
- Losing the reference to the rest of the list before relinking (always save `next` first in reversal).
- Off-by-one errors in "n-th node" problems — use a dummy node and careful pointer offsets.
- Not handling edge cases: empty list, single node, k larger than list length (use `k % length`).
- Infinite loops in circular lists — always have a clear termination condition (e.g., `curr == head`).
- In DLL operations, forgetting to update both `next` and `prev`.

12. Suggested Practice Order (Easy → Hard)

1. Traversal, insertion, deletion basics
2. Reverse a linked list (iterative + recursive)
3. Find middle / Nth from end
4. Detect cycle → find cycle start → remove cycle
5. Merge two sorted lists → Sort list (merge sort)
6. Palindrome check
7. Intersection of two lists
8. Add two numbers

9. Rotate list / Swap pairs / Reverse k-group
 10. Copy list with random pointers
 11. Flatten multilevel list
 12. Josephus problem (circular list)
-

Tip: Draw the list on paper with boxes and arrows before coding — nearly every linked list bug comes from losing a pointer reference mid-operation.